

武汉大学

《现代数据库系统》实验报告

“寻根溯源”族谱管理系统设计与实现

姓 名：刘鼎琨

学 号：2023302121009

专 业：计算机科学与技术

学 院：计算机学院

指导教师：刘斌老师

二〇二六年 四月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：刘鼎琨、郑慧琳

日期： 2026 年 4 月 19 日

摘要

本报告设计并实现了一个面向十万级数据规模的数字化族谱管理系统数据库。针对族谱数据复杂的树状与网状关联特征，系统采用了“重数据库计算、轻应用层处理”的架构方案。在逻辑设计阶段，各关系模式均经过数学推导，证明满足第四范式（4NF），以避免数据冗余与操作异常。在物理实现层面，系统基于 PostgreSQL 引擎，通过编写 PL/pgSQL 触发器与声明式约束，在数据库底层处理防断链保护、生卒时序校验、配偶双向同步及全族代际平移等业务逻辑，以保证数据一致性。针对深层家族树遍历与残缺档案模糊检索的查询需求，系统部署了带有条件过滤的部分复合 B+ 树索引并结合 pg_trgm 扩展的 GIN 倒排索引，并使用 Recursive CTE（递归公用表表达式）在数据库内部完成血缘寻路操作。测试结果表明，在装载十万级生成数据的场景下，系统能够正确维护亲属关联约束，顺利完成宏观统计、祖先溯源及最短亲缘链路计算，验证了该架构在大规模关系型数据管理中的可行性与实际执行效率。

关键词：数字化族谱；PostgreSQL；触发器；Recursive CTE；索引优化；范式证明

ABSTRACT

This report details the design and implementation of a database for a digital family tree management system, scaled for 100,000-level records. Addressing the complex tree and network structures inherent in genealogy data, the system adopts a "heavy-database, light-application" architecture. During the logical design phase, all relational schemas are proven to satisfy the Fourth Normal Form (4NF) through mathematical derivation to avoid data redundancy and operation anomalies. At the physical implementation level, the system uses PostgreSQL features by deploying PL/pgSQL triggers and declarative constraints to handle business logic—including broken-link prevention, birth-death chronological validation, bidirectional spouse synchronization, and clan-wide generation shifting—to maintain data consistency. To address the queries of deep family tree traversals and fuzzy text retrieval for incomplete archives, the system deploys conditional B-Tree partial/composite indexes and GIN inverted indexes with the `pg_trgm` extension, and uses Recursive Common Table Expressions (CTE) to process kinship routing operations within the database. Testing shows that under a 100,000-record baseline, the system correctly maintains relative connections and completes macroscopic generation statistics, ancestral tracing, and shortest kinship path calculations, validating the feasibility and query efficiency of this architecture in managing large-scale relational data.

Keywords: Digital Genealogy; PostgreSQL; Triggers; Recursive CTE; Index Optimization; Normalization Proof

目 录

原创性声明	II
摘 要	1
ABSTRACT	2
1 项目背景与需求分析	1
1.1 项目背景	1
1.2 核心挑战与解决思路	1
1.3 业务处理需求分析	2
2 概念设计	5
2.1 实体与属性的提取	5
2.2 联系分析与局部 E-R 图设计	5
2.3 全局 E-R 图	7
3 逻辑与物理设计	9
3.1 表的转换生成	9
3.2 范式分析	10
3.3 约束设计	13
3.3.1 基础声明式约束设计	13
3.3.2 触发器设计	14
3.4 索引设计	15
4 项目技术设计	17
4.1 项目架构与技术选型	17
4.1.1 系统架构	17

4.1.2 主要技术栈及版本	18
4.2 核心 SQL 设计与实现	19
4.3 索引设计性能分析	22
4.4 创新亮点：AI 辅助检索与高级检索	24
5 运行过程及结果.....	26
5.1 测试数据生成	26
5.2 功能展示与运行效果.....	27
5.2.1 宏观统计与图表展示	27
5.2.2 成员录入与底层约束拦截	28
5.2.3 递归查询与家族树渲染	29
5.2.4 亲缘链路计算与索引寻址	30
6 个人工作内容及感想	31

1 项目背景与需求分析

1.1 项目背景

“寻根溯源”族谱管理系统是一个支持多用户协同的族谱信息管理平台。系统需要管理多个家族的族谱信息（如谱名、姓氏、修谱时间等），并详细记录家族成员的个人属性（姓名、性别、生卒年、生平简介）以及错综复杂的血缘（父子、母子）与婚姻（配偶）关系。根据实验的大数据量场景要求，系统需要具备支持海量数据存储与检索的能力。设定的数据基线为全系统不少于 10 万条成员记录，且至少包含一个承载超 5 万名成员、繁衍跨度达 30 代以上的大型单一族谱。

1.2 核心挑战与解决思路

在十万级元组的关系型数据库中映射并操作图状的家族拓扑网络，主要面临实验要求中明确提出的四个核心技术挑战。

首先是成员层级关系深度不确定的问题。在追溯成员历代祖先或直系后代时，查询的层级深度在预编译期是未知的。传统的多表关联操作极易引发深度嵌套与全表扫描。为了解决这一问题，本系统采用数据库原生的递归公用表表达式（Recursive CTE），将树状结构的递归解析下推至数据库执行引擎，从而规避了应用层多次循环查询产生的额外网络 I/O 开销。

其次是海量数据下的同名歧义与逻辑断链风险。多用户协同修谱且数据量庞大时，异代同名同姓现象频繁，且人工录入极易引发长幼生卒年倒置、跨辈分结婚、删除中间节点导致家族树断裂等逻辑错误。针对这一难点，系统在数据库底层全面引入了 CHECK 约束与行级触发器。在数据写入阶段进行强制拦截与时序校验，例如严格限制父亲的出生年份必须早于子女，以此保障底层数据的绝对一致性。

第三个挑战在于层级溯源与模糊检索的性能瓶颈。系统要求在十万级数据中快速查询分支后代，并支持人名的模糊查找。如果不加干预，这些高频操作极易导致数据库进行缓慢的全表扫描。因此，系统针对父子关系跳转设计了 B+ 树复合索引以加速递归过程；针对姓名模糊匹配，引入了 GIN 倒排索引结合

pg_trgm 扩展，成功突破了传统 B 树在包含前导通配符的模糊查询下的索引失效问题。

最后是亲缘关系通路的判定开销过大。当输入任意两个人物 ID 查询其亲缘连通性及最短链路时，本质上是在关系型表中实现图论搜索算法，在十万级数据下计算极其耗时。为降低计算延迟，系统通过合理设计自引用的外键约束（father_id, mother_id, spouse_id）构建了双向复合关联图，并配合底层索引将大范围的无关数据过滤在内存搜索之前，确保了复杂拓扑查询的快速响应。

1.3 业务处理需求分析

族谱管理系统的具体功能要求，主要可以分为下面这几部分。首先是用户的登录和注册，保证能安全地使用系统。其次是实现族谱的建立和多人协作功能，用户自己能新建族谱，还能通过邀请功能，让亲戚朋友一起来修谱，共同维护一个家族的信息。第三是管理具体的家族成员信息，可以添加亲人，修改他们的生平，重要的是把他们的父母、配偶关系绑定清楚，不能搞错辈分和生卒年时间。最后是提供好用的查询和统计功能，比如直观地看到树状的家族图谱，能往上查找历代祖宗，查找任意两个人之间的亲缘关系，还能计算家族人数，男女比例等。

我们用自顶向下的方法画了系统的数据流图（DFD）。在这个图里，系统的使用者是数据的来源和去向。系统里面有四个主要的加工过程：第一个是‘用户认证’，处理账号密码；第二个是‘族谱档案维护’，处理新建族谱和邀请协同的指令；第三个是‘族人信息登记’，接收用户填写的族人表单，还要检查辈分和时间对不对；第四个是‘图谱分析与展示’，把查到的数据算好，返回给用户看。这些数据分别存放在四个地方：‘用户信息库’、‘族谱档案集’、‘协作记录文件’和‘族人关系库’。箭头表示了数据在用户、加工和存储之间的流动情况。

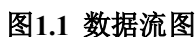


表1-1 数据项定义表

表1-2 数据结构定义表

3 / 31

数据结构名称	业务含义	包含的核心数据项
成员档案	描述个人的自然生平及亲属关联指针	成员编号，成员姓名，性别，出生年份，死亡年份，父亲编号，母亲编号，配偶编号
协作授权	描述多名用户共同维护同一族谱的权限映射	族谱编号，用户编号

表1-3 数据存储与数据流定义表

名称	类型	业务描述
用户信息库	数据存储	存放所有注册的用户信息，供用户认证过程读取与比对
族谱档案集	数据存储	存放各家族的族谱档案，供空间建立与档案维护过程调用
族人关系库	数据存储	存放成员档案，是系统绘制家族树与计算亲缘链路的数据源
协作记录文件	数据存储	存放协作授权信息，用于判定指定用户是否有权修改某本族谱
成员登记表单	数据流	由用户端发出，包含待录入的各项成员属性，流向信息登记处理环节
族谱分析结果	数据流	由图谱分析环节生成，包含经算法梳理的层级网络数据，流向用户端进行展示

2 概念设计

2.1 实体与属性的提取

根据第一章的需求分析结果，我们把系统中流动的数据抽象成具体的概念模型。采用自底向上的方法，我们首先从数据字典中提取出了系统需要管理的核心实体，并确定了它们各自的属性。

我们提取的第一个核心实体是“用户”，它代表了使用我们系统的真实自然人。这个实体包含的属性有用户编号、用户名、密码凭据和电子邮箱。其中，用户编号是用来唯一代表某一个用户的标识。第二个提取出来的实体是“族谱”，用来表示一个个独立的家族档案集。它包含的属性有族谱编号、族谱名称、核心姓氏和创建时间。同样地，族谱编号是该实体的唯一标识。第三个也是数据量最大的实体是“成员”，用来表示族谱里面的具体亲人个体。这个实体包含的属性非常丰富，具体包括成员编号、姓名、性别、出生年份、死亡年份、世代序数以及生平简介。成员编号是这个实体身上的唯一标识。

2.2 联系分析与局部 E-R 图设计

找出核心实体之后，我们开始分析这些实体之间在日常修谱业务中有什么逻辑联系，并据此设计了三个局部的 E-R 图。

首先是分析用户和族谱这两个实体之间的联系。在我们的系统中，一个用户可以自己建立多本不同的族谱，但一本具体的族谱最初只能由一个特定的用户来创建，这就构成了一个一对多的“创建”联系。此外，因为系统支持多人共同修谱，所以一本族谱可以邀请好几个其他用户来帮忙编辑，一个用户也可以接受邀请去帮别人修多本不同的族谱，这就又构成了一个多对多的“协作”联系。这两个联系虽然都发生在用户和族谱之间，但代表了不同的业务意义，前者代表所有权，后者代表编辑权。

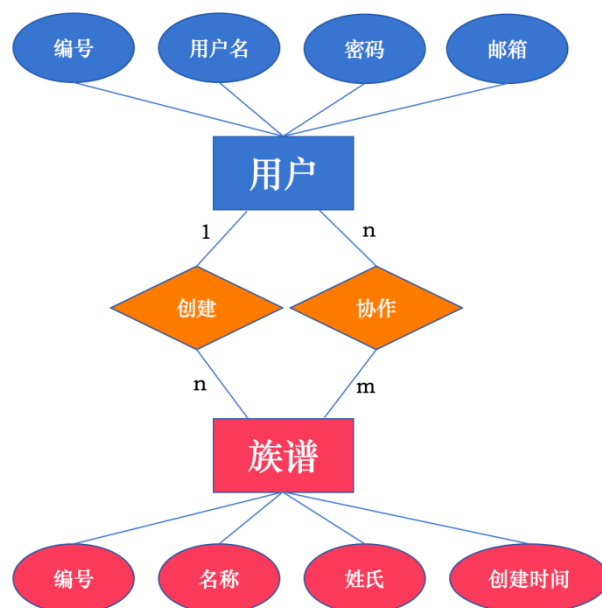


图2.1 用户-族谱局部E-R图

其次是分析族谱和成员之间的联系。这个逻辑非常明确，一本族谱里面会详细记录成百上千个家族成员，但在我们系统的设定中，一个具体的成员记录只能归属于某一本特定的族谱，不能同时跨越在两本不同的族谱里。因此，族谱和成员实体之间是一个标准的、一对多的“包含”联系。

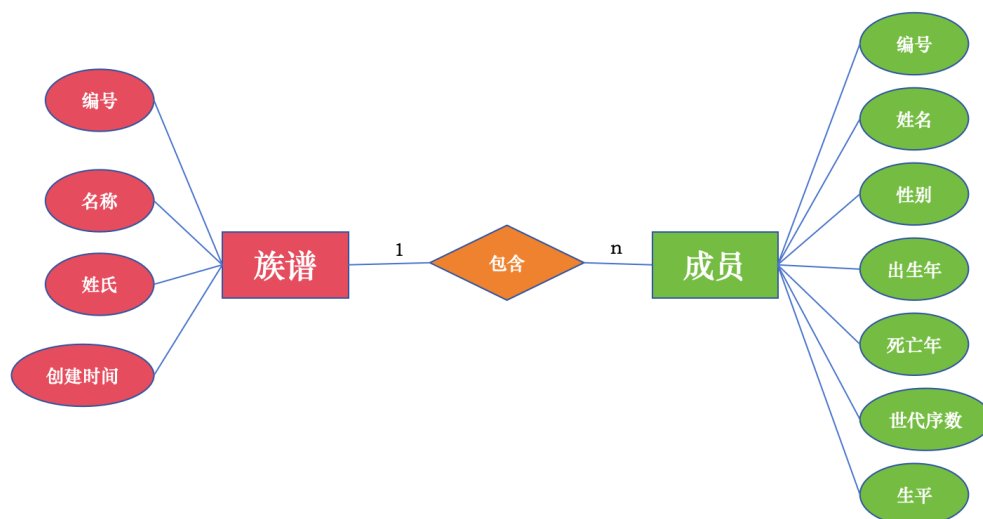


图2.2 族谱-成员局部E-R图

最后是分析成员实体自己内部的自联系，这也是体现家族血缘网络最关键的一步。在垂直的血缘关系上，系统规定一个人只能有一位亲生父亲和一位亲生母亲，但一位父亲或母亲可以生育多个不同的子女，这就使得成员实体内部

构成了两个独立的一对多“父亲”联系和“母亲”联系。在水平的婚姻关系上，系统当前只记录一个人的一名合法配偶，这就构成了成员实体内部的一个一对一“配偶”联系。

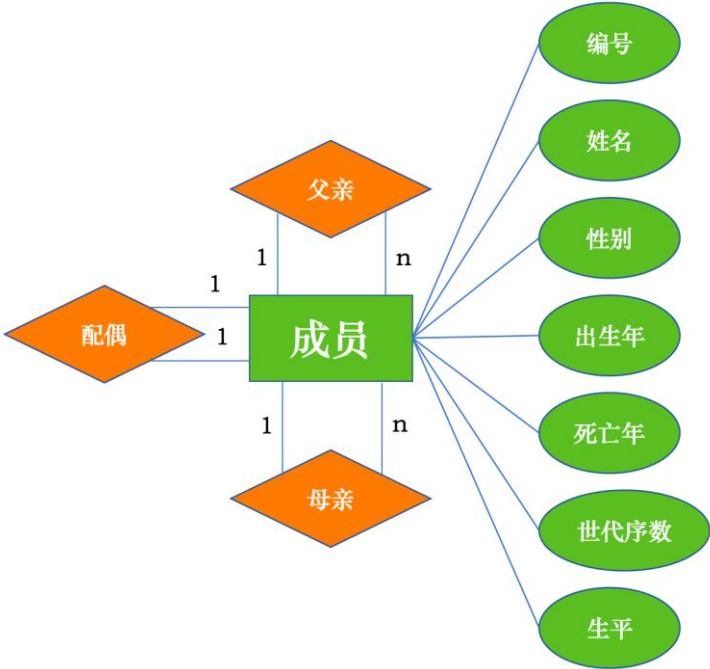


图2.3 成员局部E-R图

2.3 全局 E-R 图

在完成上述各个局部视图的构思后，我们将这三张局部的 E-R 图合并，生成系统最终的全局基本 E-R 图。在合并图纸的过程中，我们排查了各个局部视图之间是否存在冲突。由于本系统的所有实体和属性都是我们在前期统一规划命名的，因此在合并时没有遇到同名异义或者异名同义的命名冲突，各个属性的数据类型也保持了一致。同时，我们也仔细检查了实体间的联系，确认保留下来的创建、协作、包含以及亲属自联系都是业务必须要用的，不存在可以被相互推导出来的冗余联系。最终集成后的全局基本 E-R 图如下所示。

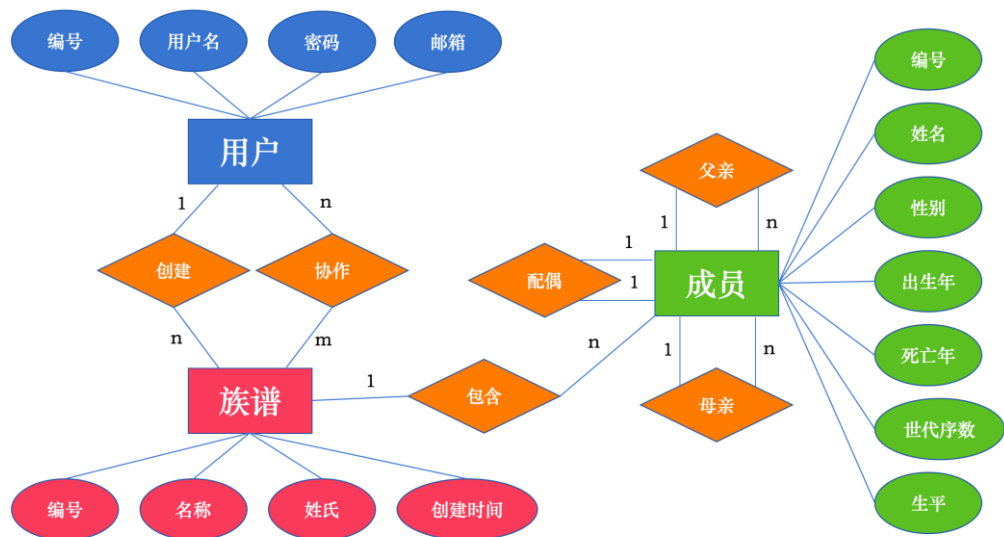


图2.4 全局基本E-R图

3 逻辑与物理设计

3.1 表的转换生成

根据第二章画好的全局 E-R 图，我们把概念模型转换成关系模式，即数据库里能直接建的表。首先，图里的“用户”、“族谱”和“成员”这三个实体，都可以直接转换成一张表，这就有了 Users、Clans、Members 三张基础表。其次，我们来处理它们之间的联系。对于用户创建族谱、族谱包含成员这两个一对多的联系，按照转换规则，我们不需要新建表，只需要把“一”那一端的主键拿过来，塞到“多”那一端的表里当外键就行，因此对于用户创建族谱，在族谱表中加入这个 creator_id，表示创建者编号，关联 Users 表的 user_id。对于族谱包含成员这个联系，则在成员表中加入 clan_id，表示所属族谱编号，关联 Clans 表的 clan_id。成员自己内部的父亲、母亲和配偶联系也同理，直接在成员表里加上对应的外键字段。最后，对于用户和族谱之间多对多的“协作”联系，按照规则必须单独抽出来建一张新表(Clan_Collaborators)。这张新表的主键就是把用户的主键和族谱的主键拼在一起组成的。经过转换，3 个实体变成 4 张表，具体的表结构如下：

表3-1 用户表 (Users)

字段名	数据类型	主/外键	字段说明与约束
user_id	SERIAL	主键	用户的唯一编号，系统自增
username	VARCHAR(255)		用户名，必须唯一且不能为空
password_hash	VARCHAR(255)		加密后的密码，不能为空
email	VARCHAR(255)		用户的电子邮箱地址

表3-2 族谱表 (Clans)

	数据类型	主/外键	字段说明与约束
clan_id	SERIAL	主键	族谱的唯一编号，系统自增
clan_name	VARCHAR(255)		族谱的对外展示名称，不能为空
surname	VARCHAR(255)		该族谱的核心姓氏，不能为空
revision_time	TIMESTAMP		最后修改时间，默认获取当前时间
creator_id	INT	外键	创建者编号，关联 Users 表的 user_id

表3-3 成员表 (Members)

字段名	数据类型	主/外键	字段说明与约束
member_id	SERIAL	主键	成员的唯一编号，系统自增
name	VARCHAR(255)		成员真实姓名，不能为空
gender	CHAR(1)		性别，约束只能填写 'M' 或 'F'
birth_year	INT		出生年份，不能为空
death_year	INT		死亡年份（可为空，必须大于等于出生年份）
bio	TEXT		成员生平简介长文本
generation	INT		世代序数（用于记录该成员的辈分排位）
clan_id	INT	外键	所属族谱编号，关联 Clans 表的 clan_id
father_id	INT	外键	父亲编号，关联 Members 表自身的 member_id
mother_id	INT	外键	母亲编号，关联 Members 表自身的 member_id
spouse_id	INT	外键	配偶编号，关联 Members 表自身的 member_id

表3-4 协作表 (Clan_Collaborators)

字段名	数据类型	主/外键	字段说明与约束
clan_id	INT	联合主键、外键	参与协作的族谱编号，关联 Clans 表的 clan_id
user_id	INT	联合主键、外键	被授权协作的用户编号，关联 Users 表的 user_id

（注：该表为关联实体，采用复合主键）

3.2 范式分析

我们通过数学推导确保系统中的四张表能够彻底消除数据冗余和各类操作异常。我们将采用分步推导的方法，从第一范式开始，逐级证明各表达到第四范式的要求。

首先证明用户表（Users）。该关系 $R_{Users}(U_1, F_1)$ 包含的属性集 U_1 与非平凡函数依赖集 F_1 分别如下：

$$U_1 = \{user_id, username, password_hash, email\}$$

$$F_1 = \{user_id \rightarrow U_1, username \rightarrow U_1\}$$

从第一范式（1NF）来看，表内每个分量（字段）均为不可分的数据项，因此满足 1NF。在分析更高范式前，需注意根据业务约束，该表实际存在两个候选键：user_id（系统自增编号）和 username（要求唯一）。由于该关系的候

选键均为单属性，根据推论，若关系模式的候选键是单属性的，则必定是 2NF，这说明表中天然不存在非主属性对候选键的部分函数依赖。在 3NF 层面，所有非主属性（如 password_hash、email）均直接依赖于候选键，没有任何一个非主属性传递函数依赖于候选键，故满足 3NF。

进一步考察 BCNF，根据定义，若函数依赖集合中的所有非平凡函数依赖的左部都包含候选键，则符合 BCNF。在 F_1 中，所有的决定因素（user_id 或 username）本身就是候选键，因此满足 BCNF。最后检验 4NF，在用户关系中，每个用户仅对应唯一的密码和邮箱，不存在一个属性对应多组独立且无关的属性集合的情况；换言之，对于任何一个非平凡多值依赖，其左部都包含了候选键，因此该表自然满足 4NF。

其次证明族谱表（Clans）。该关系 $R_{Clans}(U_2, F_2)$ 的属性集 U_2 与非平凡函数依赖集 F_2 分别如下：

$$U_2 = \{clan_id, clan_name, surname, revision_time, creator_id\}$$

$$F_2 = \{clan_id \rightarrow clan_name, clan_id \rightarrow surname, clan_id \rightarrow revision_time, clan_id \rightarrow creator_id\}$$

该表满足 1NF 是显而易见的。其候选键为单属性 clan_id，因此消除了部分依赖，满足 2NF。在族谱信息中，姓氏、名称等属性只与族谱 ID 直接相关，不存在非主属性间的内部依赖，故消除了传递依赖，满足 3NF。同时，因为 F_2 中所有的决定因素都包含了候选键，该模式满足 BCNF。在多值依赖分析中，一个族谱 ID 仅描述一个特定的家族名称空间，不涉及如“一个族谱对应多个独立校训且对应多个独立发源地”这种多值关联逻辑，不存在非平凡多值依赖，故符合 4NF。

接着证明核心的成员表（Members）。该关系模式 $R_{Members}(U_3, F_3)$ 包含的属性集 U_3 与非平凡函数依赖集 F_3 分别如下：

$$U_3 = \{member_id, name, gender, birth_year, death_year, bio, generation, clan_id, father_id, mother_id, spouse_id\}$$

$$F_3 = \{member_id \rightarrow U_3 - \{member_id\}\}$$

在第一范式（1NF）的考量下，表内所有字段均为原子数据类型且不可再分，因此满足 1NF。关于候选键的确定，虽然业务要求姓名（name）不能为空，但在同一族谱内完全可能出现重名情况，姓名本身不具备全局唯一性；此

外，`father_id`、`mother_id` 和 `spouse_id` 等关联字段均允许为空（NULL），根据实体完整性要求也无法作为候选键。因此，该关系的唯一候选键为单属性 `member_id`。由于候选键由单属性组成，该模式天然不存在非主属性对候选键的部分函数依赖，故完全满足第二范式（2NF）。

在第三范式（3NF）的分析中，表中的所有非主属性均直接依赖于主键 `member_id`。外键 `clan_id` 用于标识该成员所属的族谱空间，而 `father_id` 等外键用于标识其个体的人际关联，这些非主属性之间不存在决定与被决定的函数依赖关系，因此消除了传递依赖，满足 3NF。

进一步考察 BCNF 范式，由于 F3 中唯一的决定因素是候选键 `member_id`，即所有非平凡函数依赖的左部都包含了候选键，因此该模式满足 BCNF 要求。在第四范式（4NF）的判定上，虽然一名成员在逻辑上与多名亲属（如多名子女）存在联系，但本表的物理结构仅通过单值的外键字段记录该成员与特定父母、配偶的指针，并未在单行内存储属性集合。所有的多值关系都是通过外键在多行数据之间体现的，因此该单表结构内不存在违反 4NF 的非平凡多值依赖，完全达到了 4NF 的规范化标准。

最后证明协作表（`Clan_Collaborators`）。该关系 $R_{\text{ClanCollaborators}}(U_4, F_4)$ 的属性集 U_4 与非平凡函数依赖集 F_4 如下：

$$U_4 = \{\text{clan_id}, \text{user_id}\}$$

$$F_4 = \emptyset$$

由于该表是为了实现多对多联系而设立的，其属性集合即为候选键，属于“全码”关系，且没有非主属性，因此其函数依赖集为空。根据定义，该表天然满足 1NF、2NF 和 3NF。又因为不存在任何决定因素不是候选键的非平凡函数依赖，它同样满足 BCNF。在多值依赖方面，虽然描述了一对多的协作关系，存在以下多值依赖：

$$\text{clan_id} \twoheadrightarrow \text{user_id}$$

但由于该关系的属性集仅由这两个字段组成，根据 4NF 定义，这种多值依赖属于“平凡多值依赖”，满足如下公式：

$$X \cup Y = U_4$$

平凡多值依赖并不会导致数据冗余，因此，协作表也完全符合 4NF 的规范

要求。

综上，通过对全系统四张表的数学推导，我们可以确认所有关系模式均已达到了第四范式（4NF）级别。本系统的逻辑结构设计在数理逻辑上是健壮的，在设计上不会有数据插入、删除异常及由于多值依赖引起的信息冗余。

3.3 约束设计

3.3.1 基础声明式约束设计

基础的声明式约束（包含列级约束与表级约束）主要用于处理单行数据的合法性检查以及简单的表间级联反应。这类直接写在建表语句中的物理约束执行速度最快，是我们防御错误数据落库的第一道防线。我们简要列举了一些和本次实验背景相关的设计考量，一些基础设计如主键自增且唯一等则省略没有列出，具体设计如表 3-5 所示。

表 3-5 基础声明式约束设计表

约束类型	作用对象	落地方案	约束说明与设计考量
主键自增	各表的主键字段	SERIAL 类型	系统自动分配编号，避免多人共同修谱时手工录入主键导致冲突报错。
常识拦截	性别、生卒年字段	CHECK 子句	采用列级约束规定性别只能填 'M' 或 'F'；采用表级约束校验死亡年份如果填写了，必须大于等于出生年份。将离谱的逻辑错误挡在数据库门外。
容器清理	clan_id 外键	CASCADE (级联删除)	族谱是一个大容器，当用户删除整本族谱时，数据库会自动级联删光里面成百上千的成员，保证数据空间干净。
防误删保护	父母、配偶外键	SET NULL (置空)	当误删某个长辈时，只把晚辈身上连向他的指针清空。如果这里也用级联删除，删一个老祖宗会把几千个子孙连带删光。

针对不同的外键场景，我们采取了截然不同的级联处理策略。族谱作为外层大盒子，销毁时必须连带清理内部所有归属数据；而家族成员只是网状亲属关系里的一个节点，当一个节点被删掉时，我们仅利用置空（SET NULL）策略断开连线，把剩下的孤立节点保留下来供后续人工修补。这样能最大程度保护辛辛苦苦录入的家族档案不被意外的级联删除彻底清空。

3.3.2 触发器设计

表列约束虽然执行速度快，但它只能针对单行数据进行简单校验，没法处理“跨行对比”和“树状连通性”的复杂问题。为了解决修谱业务里的深层逻辑判断以及一些自动化操作，我们设计了五组核心触发器来接管底层逻辑。去掉了具体的代码实现细节，这五组触发器在设计阶段的业务规划如表 3-6 所示。

表 3-6 触发器设计表

触发器功能名称	触发时机	拦截与处理目标
关系综合校验	BEFORE INSERT / UPDATE	对比两代人的生卒年和性别，防止时序倒转和性别错乱，并拦截跨族谱认亲。
防断链保护	BEFORE DELETE	防止中间长辈被随便删掉，导致家族树在前端画图时断开变成孤岛。
配偶双向同步	AFTER INSERT / UPDATE / DELETE	实现婚姻关系的自动化管理，单向绑定配偶后，系统自动完成另一方的双向绑定。
修订时间刷新	AFTER INSERT / UPDATE / DELETE	当族谱里的成员档案发生任何增删改动作时，自动刷新外层族谱表的最后修订时间。
全族代际平移	AFTER UPDATE	在为第一代始祖追加长辈时，由数据库底层直接进行全族辈分的下推运算，解决前端遍历计算带来的卡顿。

首先是防御性质的触发器。防断链保护触发器是为了保护家族树结构的完整性。如果在中间层级随便删掉一个既有父母又有孩子的成员，这棵树就被切断。触发器会在删除动作前去数据库里摸底，如果发现这个人上下都有节点，就会拒绝删除，强制要求用户先去处理他孩子的归属。其次是关系综合校验触发器，当用户给一个成员绑定父母时，触发器会自动顺着编号去查长辈的档案。如果发现父亲的出生年份比儿子还晚、父亲的性别填成了女性，或者这个父母根本不属于当前这本族谱，就会立刻抛出异常并终止录入，有效防御了违背常理的关系绑定。

我们还利用触发器实现了业务逻辑的自动化封装。最典型的就是配偶双向同步触发器。在现实中婚姻关系是对等的，如果把甲的配偶设为乙，那么乙的配偶也必须自动变成甲。依靠前端发送两次请求容易因为网络波动产生错误，所以我们把它下沉为底层触发器，只要用户保存了一方的配偶，数据库就会在后台自动完成另一方的双向绑定和解绑。再比如修订时间刷新触发器，由于判断一本族谱是否活跃看的是里面成员的变动情况，所以每当成员表发生增删

改，触发器就会悄悄更新主族谱表的“最后修改时间”，这样前端在查询活跃族谱时直接读取主表即可，免去了遍历全族成员的开销。

针对大规模族谱管理的性能瓶颈，系统设计了全族代际平移触发器。家族修撰经常需要向远古追溯，当用户为原本的第 1 代始祖挂载更早的长辈时，原先几万名成员的代际序数（辈分）全都需要顺延加一。如果交由前端代码提取数据、遍历计算再逐条回写，网络传输和内存开销极大。该触发器捕获到始祖变更事件后，会直接在数据库引擎内部发起一次批量的范围更新。将沉重的树状图重置逻辑下推为单次数据库原生操作，保障了系统在面对大量数据时的响应速度。

3.4 索引设计

在族谱数据的日常检索中，高频的图谱递归查祖先和按名字模糊查人极易容易引发全表扫描。为了提高数据库的响应效率，我们在底层部署了针对性的物理索引。在初期的索引选型中，我们考虑到 Hash 索引仅支持严格的等值查询，无法胜任按世代跨度的范围筛选；而对于性别这类仅有男女两个值的低区分度字段，建立普通位图索引又会被优化器直接无视，甚至在并发写入时引发页锁竞争。因此，综合考量维护成本与实际收益，我们最终确立了以 B+树为主、GIN 倒排配合 pg_trgm 扩展为辅的索引设计方案。系统核心的索引部署情况如表 3-7 所示。

表 3-7 物理索引设计表

目标表与字段	索引类型	业务场景与设计逻辑
members (clan_id, father_id) 等	B-Tree (部分复合索引)	加速递归寻址。通过附加 WHERE father_id IS NOT NULL 过滤掉无父辈的根节点，缩小索引体积，支撑 Recursive CTE 的频繁连接。
members (clan_id, generation, birth_year)	B-Tree (复合索引)	匹配多租户与统计排序。利用最左前缀原则，在隔离特定族谱的同时，直接提供按辈分和生辰排好序的数据，免去内存排序开销。
members (name)	GIN pg_trgm	突破模糊匹配瓶颈。挂载 pg_trgm 扩展，解决 B-Tree 遇到 LIKE '%关键字%' 时前导通配符导致索引失效的问题。

为了解决关系型数据库在处理深层树状结构时的遍历痛点，我们设计了带有条件过滤的部分复合索引。当用户触发“向上追溯某成员历代祖先”的查询

时，底层需要通过 Recursive CTE（递归公用表表达式）逐层循环比对长辈的编号。如果直接建单列索引，效率依然不够高。我们在代码中实际部署的是诸如 (clan_id, father_id) WHERE father_id IS NOT NULL 这样的部分索引。它不仅将族谱编号和父亲编号组合在一起加快联合查询，更妙的是利用 WHERE 子句直接在物理层面上剔除了那些没有记录父亲的“始祖”节点。这极大地压缩了 B+树的体积，使得查询引擎在执行深度遍历时，能够利用更小的树结构进行精确的指针跳转。

对于宏观的统计查询需求，我们部署了 (clan_id, generation, birth_year) 复合索引。由于多租户架构决定了所有业务查询必须先施加族谱编号的前置过滤，我们将 clan_id 放在复合索引的第一顺位。紧接着将世代序数和出生年份放在其后，这保证了引擎在圈定特定家族的数据页后，提取出的节点记录在物理磁盘上已经严格按辈分和年龄排好了序。当系统需要筛选特定代际的数据时，数据库可以直接沿索引顺序扫描并提取，彻底规避了耗时的内部重新排序动作。

针对族谱残缺档案中高频出现的模糊查人需求，我们引入了 GIN（通用倒排索引）联合 pg_trgm 扩展的重构设计。常规的 B+树索引严格依赖“最左前缀匹配”原则，当 SQL 语句的匹配模式以通配符开头时（例如 LIKE '%阳修%'），执行引擎无法预判首字符，只能放弃索引去挨个扫描全表数据。为打破这一限制，我们启用了 PostgreSQL 的 pg_trgm 扩展。在数据写入阶段，执行引擎会利用长度为 3 的滑动窗口，将成员姓名切分为多个连续的定长词元（Trigram Token，例如把“欧阳修”解构为“欧阳”、“阳修”等切片）。

切分完成后，GIN 倒排索引接管这些离散的词元，构建出一个“词元指向物理行指针（TID）”的反向映射字典。在查询执行阶段，当应用层下发模糊查询指令时，引擎首先将前端传入的搜索词同样切分成词元，然后直接切入 GIN 倒排字典，命中对应的索引条目。随后，引擎提取字典中关联的物理行指针，并在内存中执行位图交集运算以滤除冗余记录。最终基于高纯度的指针集合发起回表操作，直接精确定向地读取底层物理数据页。这一架构将传统的“字符串逐字对比”重构为了“字典寻址与位图聚合”，从底层物理逻辑上解决了模糊查询的性能瓶颈。

4 项目技术设计

4.1 项目架构与技术选型

4.1.1 系统架构

我们的项目采用了标准的前后端分离架构。前端用 Vue 3 和 ECharts 负责画页面和渲染族谱树，后端用 ASP.NET Core 8 提供基础接口，底层数据则存在 PostgreSQL 16 里。为了让这套系统在任何电脑上都能一键跑起来，我们用 Docker Compose 把它们打包成了统一的容器环境。具体架构如下图所示：

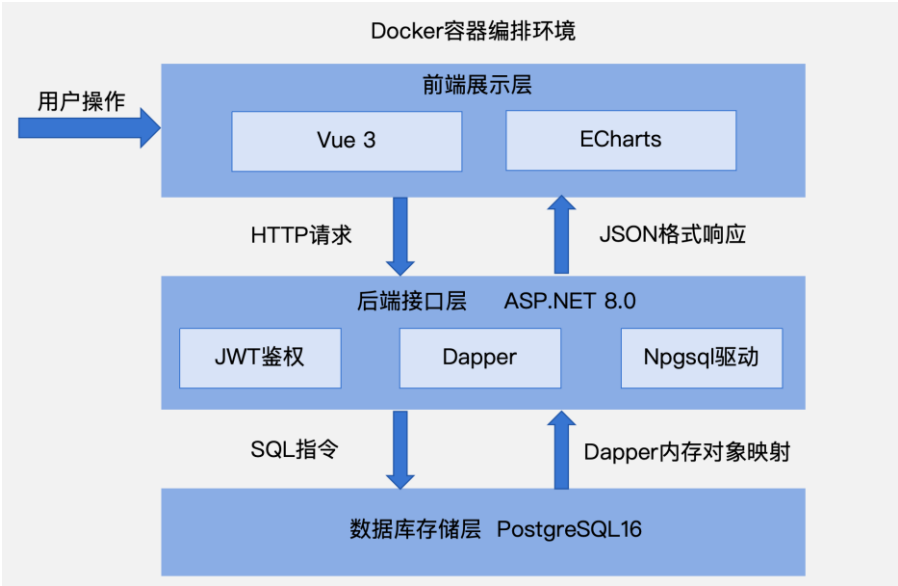


图4.1 系统架构图

系统运行时，前端通过 HTTP 请求访问后端接口，后端连接 PostgreSQL 数据库完成数据读写。数据库中保存用户、族谱、成员及协作关系等核心数据。项目结构清晰，便于按模块进行开发和维护。以下是项目中使用到的技术：

表 4-1 项目架构表

层次	主要功能	采用技术
前端	页面展示、数据交互、图表显示	Vue 3、Vite、TypeScript、Element Plus、Pinia、Vue Router、ECharts

层次	主要功能	采用技术
后端	接口服务、登录认证、业务处理、数据库访问	ASP.NET Core 8、Dapper、JWT、Swagger
数据库	数据存储、约束控制、索引优化、触发器实现	PostgreSQL 16
部署	容器化运行与多服务编排	Docker、Docker Compose

4.1.2 主要技术栈及版本

表4-2 项目技术选型表

模块	技术/组件	版本	作用
前端	Vue	3.4.38	构建单页应用
前端	Vite	5.4.2	前端开发与构建工具
前端	TypeScript	5.5.4	提供静态类型支持
前端	Vue Router	4.4.3	前端路由管理
前端	Pinia	2.1.7	状态管理
前端	Element Plus	2.8.1	页面组件库
前端	ECharts	5.5.1	图表展示
前端	Vue ECharts	7.0.3	ECharts 的 Vue 封装
后端	.NET	8.0	后端运行平台
后端	ASP.NET Core	8.0	Web API 框架
后端	Dapper	2.1.35	执行 SQL 并访问数据库
后端	Npgsql	8.0.4	PostgreSQL 驱动
后端	JWT Bearer Authentication	8.0.8	用户身份认证
后端	Swashbuckle.AspNetCore	6.6.2	Swagger 接口文档
数据库	PostgreSQL	16	关系数据库管理系统

模块	技术/组件	版本	作用
数据库	pg_trgm	内置扩展	支持模糊匹配索引
部署	Docker	29.4	容器化运行
部署	Docker Compose	29.4	多服务编排
容器镜像	postgres	16	数据库服务镜像
容器镜像	node	20-alpine	前端构建镜像
容器镜像	nginx	1.27-alpine	前端静态资源服务
容器镜像	mcr.microsoft.com/dotnet/sdk	8.0	后端构建镜像
容器镜像	mcr.microsoft.com/dotnet/aspnet	8.0	后端运行镜像

4.2 核心 SQL 设计与实现

实验要求中的 5 条核心 SQL 主要对应成员关系查询、递归祖先查询和统计分析查询。

给定一个成员 ID，查询其配偶及所有子女时，因为配偶关系保存在 spouse_id 中，子女关系通过 father_id 和 mother_id 建立，所以这条查询本质上就是对 members 表做自连接，再把满足父母条件的子女筛选出来。具体 SQL 语句如下所示：

```
SELECT related_type AS RelatedType, member_id AS MemberId, name AS Name,
gender AS Gender,
generation AS Generation, birth_year AS BirthYear, death_year AS DeathYear
FROM (
SELECT
'配偶' AS related_type,
spouse.member_id,
spouse.name,
spouse.gender,
spouse.generation,
spouse.birth_year,
spouse.death_year
FROM members self_member
JOIN members spouse ON spouse.member_id = self_member.spouse_id
```

```
WHERE self_member.clan_id = @ClanId
AND self_member.member_id = @MemberId
```

```
UNION ALL
```

```
SELECT
'子女' AS related_type,
child.member_id,
child.name,
child.gender,
child.generation,
child.birth_year,
child.death_year
FROM members child
WHERE child.clan_id = @ClanId
AND (child.father_id = @MemberId OR child.mother_id = @MemberId)
) AS family_members
ORDER BY CASE related_type WHEN '配偶' THEN 0 ELSE 1 END, generation,
birth_year, member_id;
```

输入成员 A 的 ID，递归查询其所有历代祖先时，祖先层级是不固定的，所以这里使用递归公用表表达式。实际实现中，后端先递归得到所有祖先成员编号，再根据编号查询节点和边信息。递归起点是目标成员本身，后续每一层都沿着 father_id 和 mother_id 向上追溯，直到没有更高层祖先为止。具体 SQL 语句如下所示：

```
WITH RECURSIVE ancestor_ids(member_id) AS (
SELECT m.member_id
FROM members m
WHERE m.member_id = @MemberId
UNION
SELECT parent_ids.parent_id
FROM ancestor_ids tree
JOIN members current_member ON current_member.member_id = tree.member_id
JOIN LATERAL (
VALUES (current_member.father_id), (current_member.mother_id)
) AS parent_ids(parent_id) ON parent_ids.parent_id IS NOT NULL
)
SELECT member_id
FROM ancestor_ids;
```

统计某个家族中平均寿命最长的一代人时，这里直接按代数 generation 分组，再计算每一代的平均寿命 AVG(death_year - birth_year)，最后按平均值降序

排列并取第一条。因为成员表本身已经有 generation、birth_year 和 death_year 这几个字段，所以这类统计可以直接在数据库层完成，不需要额外中间表。具体 SQL 语句如下所示：

```
SELECT
generation AS Generation,
AVG(death_year - birth_year)::DOUBLE PRECISION AS AverageLifespan,
COUNT(*)::INT AS MemberCount
FROM members
WHERE clan_id = @ClanId
AND death_year IS NOT NULL
GROUP BY generation
ORDER BY AverageLifespan DESC, generation ASC
LIMIT 1;
```

查询所有年龄超过 50 岁且没有配偶的男性成员时，年龄按“当前年份减出生年份”计算。筛选条件包括男性、无配偶、年龄大于 50 岁以及限定在某个族谱中。这条查询属于常规条件筛选，逻辑比较清楚。具体 SQL 语句如下所示：

```
SELECT
m.member_id AS MemberId,
m.name AS Name,
m.gender AS Gender,
m.generation AS Generation,
m.birth_year AS BirthYear,
m.death_year AS DeathYear,
spouse.name AS SpouseName
FROM members m
LEFT JOIN members spouse ON spouse.member_id = m.spouse_id
WHERE m.clan_id = @ClanId
AND m.gender = 'M'
AND m.spouse_id IS NULL
AND DATE_PART('year', AGE(CURRENT_DATE, MAKE_DATE(m.birth_year, 1, 1))) > 50
ORDER BY m.birth_year, m.generation, m.member_id;
```

找出家族中出生年份早于该辈分平均出生年份的所有成员时，思路是先在子查询中按代数分组，计算每一代的平均出生年份，再把这个结果和成员表连接，最后筛选出出生年份小于本代平均值的成员。我们设计的查询同时用到了分组、聚合和连接，比较适合放在数据库里直接完成。具体 SQL 语句如下所示：

```
WITH generation_birth_average AS (
SELECT
```

```

generation,
AVG(birth_year)::DOUBLE PRECISION AS AverageBirthYear
FROM members
WHERE clan_id = @ClanId
GROUP BY generation
)
SELECT
m.member_id AS MemberId,
m.name AS Name,
m.gender AS Gender,
m.generation AS Generation,
m.birth_year AS BirthYear,
avg_birth.AverageBirthYear AS GenerationAverageBirthYear,
m.death_year AS DeathYear,
spouse.name AS SpouseName
FROM members m
JOIN generation_birth_average avg_birth ON avg_birth.generation = m.generation
LEFT JOIN members spouse ON spouse.member_id = m.spouse_id
WHERE m.clan_id = @ClanId
AND m.birth_year < avg_birth.AverageBirthYear
ORDER BY m.generation, m.birth_year, m.member_id
OFFSET @Offset
LIMIT @PageSize;

```

4.3 索引设计性能分析

本项目中，索引设计主要对应两类查询需求。一类是根据姓名进行模糊检索，另一类是根据父节点继续向下查找子节点。前者对应 `members.name` 上的 `pg_trgm + GIN` 索引，后者对应 `members` 表上的复合索引。考虑到实验要求里明确提出要比较“查询某曾祖父的所有曾孙（四代查询）”在有无索引情况下的性能差异，因此我们重点分析的是父母关系索引在递归查询中的表现。

我们对比了三种方案。第一种是无索引临时表方案，也就是先把当前族谱的数据拷贝到临时表中，不建立任何额外索引，再在临时表上执行递归查询。第二种是位图扫描风格方案，在临时表上分别建立 `father_id` 和 `mother_id` 单列索引，希望让规划器采用 `Bitmap Index Scan`、`Bitmap Heap Scan` 或 `BitmapOr` 之类的执行方式。第三种是 `B+ 树复合索引` 方案，建立的 `(clan_id, father_id)` 和 `(clan_id, mother_id)` 复合索引在原始 `members` 表上执行递归查询。具体测试结果如下表所示。

表4-3 不同索引性能对比

方案	查询特点	执行时间
无索引临时表	临时表递归查询，不建立索引	45.771 ms
位图扫描	临时表上建立 father_id / mother_id 单列索引	94.374 ms
B+树复合索引	建立复合索引，在 members 原表上递归查询	0.124 ms

从结果上看，三种方案差异比较明显。生产索引方案最快，执行时间只有 0.124 ms；无索引临时表方案执行时间为 45.771 ms；位图扫描风格方案最慢，达到 94.374 ms。具体运行结果如下图所示：

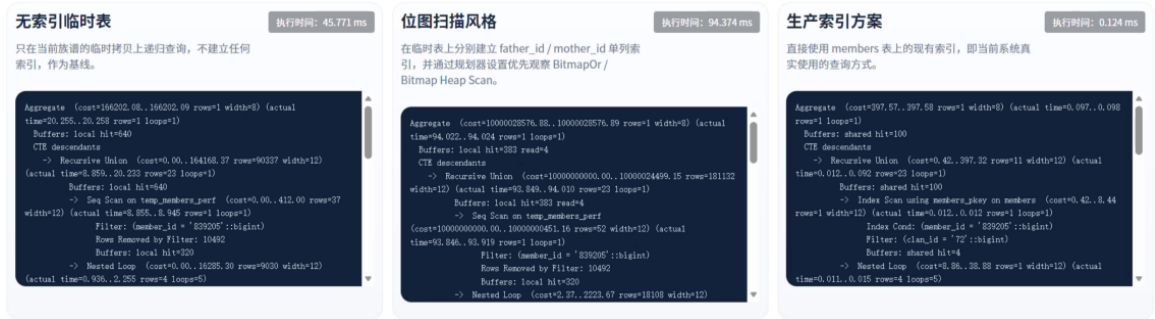


图4.2 各索引方案实际运行结果

对于无索引临时表方案。上图里可以看到，它的执行计划主要依赖 Seq Scan。这说明数据库在递归过程中查找下一层子节点时，没有办法通过索引直接定位记录，只能扫描临时表，再根据 father_id 或 mother_id 过滤。因为递归查询本身会重复执行多轮“找子节点”的动作，顺序扫描会在每一轮都付出较大的代价。

对于位图扫描方案。这个思路在某些场景下是有效的，尤其是当单列条件命中范围较大、但仍有筛选价值时，位图扫描可能会比纯顺序扫描更好。位图方案虽然减少了部分扫描，但它本身也有构建位图、合并位图和回表的开销，在递归层数不高、单次命中记录数也不算特别大的情况下，这部分开销反而可能超过它带来的收益。所以最后出现了“建了索引但更慢”的结果。

生产索引方案更贴近实际查询条件。通过建立索引 (clan_id, father_id) 和 (clan_id, mother_id)，数据库在查找子节点时，可以先把扫描范围限定在指定族谱内部，再根据父母编号定位记录。这样一来，索引过滤的范围会比单列索引更小，也更稳定。且 idx_members_clan_mother 这类索引已

经直接参与执行，这说明数据库规划器确实选择了当前系统中最匹配的访问路径。因此，最终我们选用的是 B+树复合索引。

4.4 创新亮点：AI 辅助检索与高级检索

为了满足不同水平人群的使用需求，本项目的查询模块主要包含了两大创新设计：面向普通用户的“自然语言转 SQL”人工智能辅助检索，以及面向专业用户的原生 SQL 高级检索。

首先是 AI 辅助检索功能。用户在前端独立的查询页面中，只需输入日常的大白话（例如“帮我查一下活过 80 岁的人”），系统就会将这段自然语言连同当前族谱的编号发送给后端。后端接入了兼容的大语言模型接口（采用 qwen-plus），要求模型充当“数据库翻译官”，将用户的自然语言精准转化为对应的 SQL 查询指令。为了保证系统的绝对安全，我们并没有将数据库的执行权盲目交托给大模型，而是设计了一套严密的合规校验机制。模型生成的 SQL 返回后，必须先经过后端的正则表达式拦截器。系统会显式封杀所有包含 insert、update、drop、alter 等非只读关键字的危险指令，确保最终进入数据库的只有纯粹的 SELECT 查询。

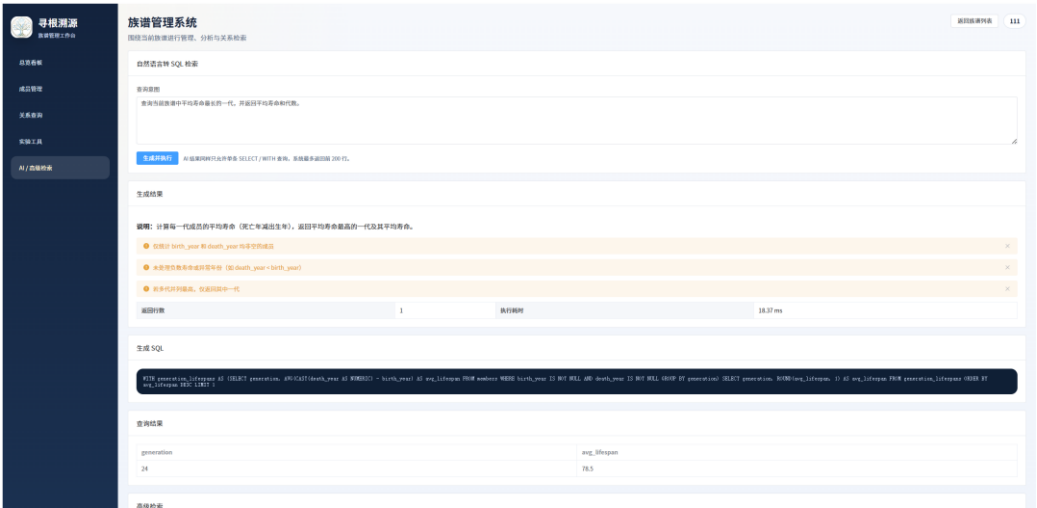


图4.3 AI查询功能示意图

其次，在 AI 查询的数据隔离方案上，我们做了一个深度的底层优化。单靠在提示词里警告大模型“只准查当前族谱”是非常不可靠的。因此，在真正执行 AI 生成的合规 SQL 之前，后端代码会利用当前的族谱编号，在数据库中

动态创建限制了数据范围的“临时视图（Temporary View）”。大模型生成的 SQL 实际上是在这个被阉割过的临时视图上运行的，这就从物理沙箱的层面上彻底杜绝了跨家族越权查询的可能。此外，如果模型生成的 SQL 存在语法瑕疵导致执行报错，系统还会自动截获错误日志，并连同原 SQL 一起发回给大模型要求其纠错重写，提升了自然语言查询的最终成功率。

除了利用 AI 照顾普通用户，系统还额外开发了面向数据库管理员的高级检索模块。这个模块非常直接，就是支持专业人员手动敲入原生 SQL 语句来进行深度查询。家族图谱本质上是一个极为复杂的无向连通图，当我们需要执行诸如“向上追溯某成员的历代直系祖先”、“向下展开某个分支的完整子嗣树”，甚至是“计算两个边缘成员之间的最短亲缘通路”时，往往需要写出包含递归公用表表达式（Recursive CTE）和多重自连接的超长语句。通过开放原生 SQL 执行窗口，不仅让专业用户能够以极客的方式拿到极其苛刻的数据，也非常方便我们演示系统中那些高难度的关系代数查询语句。

5 运行过程及结果

5.1 测试数据生成

为了验证前端家族树渲染和底层递归查询在面对海量数据时的性能，我们需要在系统中装载十万级规模的测试数据。由于家族图谱有着严苛的长幼时序和亲属外键约束，通过脚本插入文字效率和质量都不高。因此，我们的系统将数据仿真的核心逻辑封装于底层的存储过程 `seed_generated_clan` 中，利用 PostgreSQL 引擎自身的运算能力来批量组装符合逻辑的族谱数据。具体的生成逻辑如图 5.1 所示。

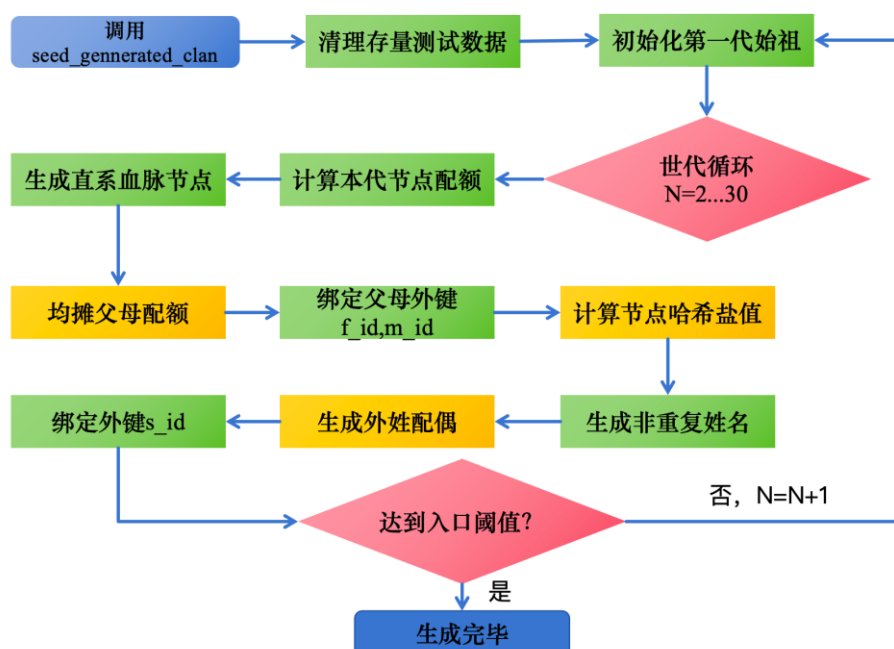


图5.1 测试数据生成逻辑图

存储过程在运行初期会利用事务机制清理掉旧的测试存量数据，保证每次压测都有一个干净的起点。在开始构建单本族谱时，系统采用了“单根向下繁衍”的模式。我们在第一代（`generation = 1`）强行生成唯一的一对始祖夫妇，把他们作为整棵家族树的绝对起点。在此基础上，存储过程才开始顺着世代序数逐代往下生成子孙。这种做法从物理层面上杜绝了“孤岛节点”的出现，保证了后续生成的几万名成员，无论顺着长辈指针怎么往上回溯，最终都一定能连通到这对始祖身上。

在具体关联家庭成员时，生成逻辑严格遵循了现实的生物学法则。每一代

生成时，系统会先挑出一部分人作为直系血亲，并给他们分配好非血缘关系的配偶，从而把平辈之间的配偶外键（`spouse_id`）连上。接着在生成下一代的孩子时，程序会把这一代的新生儿均匀地分配给上一代已经结了婚的夫妇。为了防止测试数据过于离谱，我们在分配算法里硬性限制了一对夫妻最多能生几个孩子。这就彻底避免了出现“一个人名下挂着一千个子嗣”这种既不符合常识、又会直接把前端树状图画图引擎卡死的情况。

最后，在生成成员姓名这种基础文本时，我们抛弃了极易重复的简单随机抽取法。底层专门设计了一个动态取名函数，它会根据该成员的性别和所处的代数来决定名字的风格。例如，前几代的远古祖宗会从文言文字库里组合名字，而近现代的晚辈则会使用现代字库。此外，为了彻底防止同一对父母生出两个同名同姓的孩子，生成算法在取名时还特意把父母的编号（`father_id`）也混入底层计算中。这种巧妙的机制以极低的数据库运算开销，生成了极其逼真、分散且符合代际特征的十几万条姓名数据。

5.2 功能展示与运行效果

5.2.1 宏观统计与图表展示

在系统登录后的总览看板（**Dashboard**）中，前端页面成功渲染了当前所选族谱的全局统计指标。以我们生成的这本十万级测试族谱为例，看板上清晰显示了该族谱下挂载了上万名成员，跨越了近 30 个世代。页面上的男女比例饼图和各代人数分布柱状图均做到了瞬间加载。这个极其流畅的前端表现，直接印证了我们在第三章设计的（`clan_id, generation, birth_year`）复合索引发挥了预期效能。数据库在进行上万条数据的 `GROUP BY` 分组和统计时，直接顺着物理索引就拿到了排好序的结果，没有触发拖慢速度的内存重新排序动作，保障了大屏统计图表的无阻塞渲染。

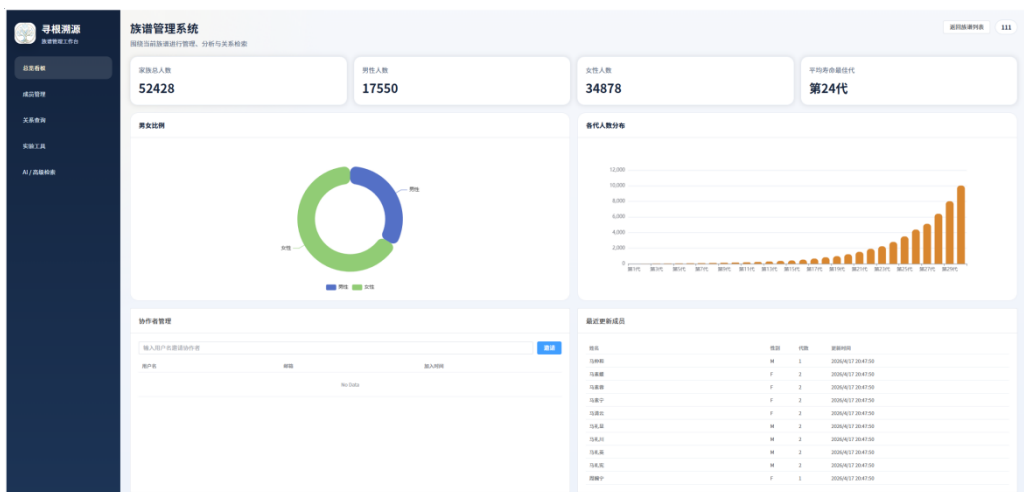


图5.2 总览看板截图

5.2.2 成员录入与底层约束拦截

在成员管理模块里，我们可以通过表单来新增或者修改一个人的档案。前端表单里的字段输入框，严格对应了数据库底层的物理结构。当我们在页面上点击保存时，前端提交的数据会直达 PostgreSQL 数据库。在测试中，如果我们故意在表单里填错信息，比如把父亲的出生年份填得比儿子还晚，或者乱填性别，我们在第三章写好的 CHECK 约束和 func_validate_member_relationships 关系校验触发器就会立刻在底层接管。数据库内核会直接拒绝这条违背常理的数据落库，并在前端抛出拦截报错。这直观地证明了我们的底层防线成功把脏数据挡在了大门外。

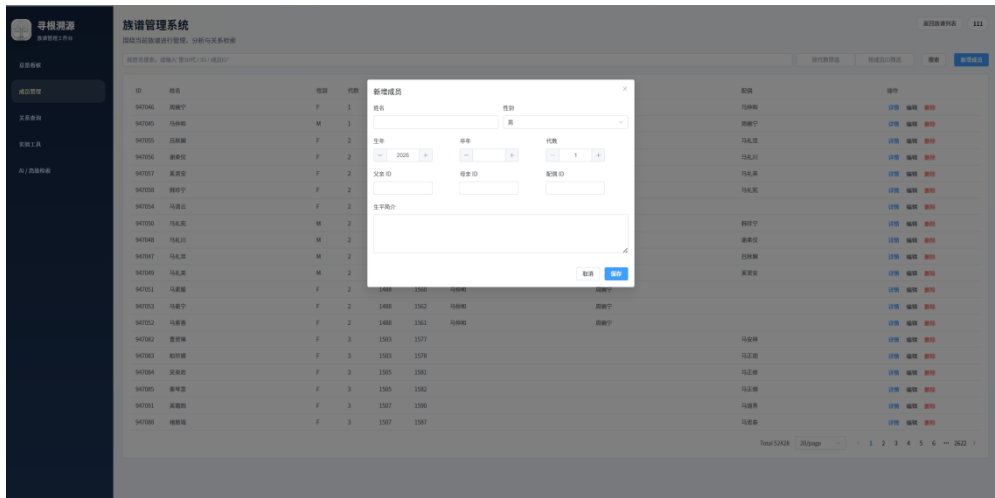


图5.3 新增成员弹窗图

5.2.3 递归查询与家族树渲染

家族系统的核心难点在于树状图谱的查询与展示。在“祖先树查询”功能中，我们随机挑了一个处于第 16 代的晚辈节点，系统画出了从他一路往上追溯到第 1 代始祖的完整单线垂直血缘路径。同时，在“分支树预览”里，系统也能以某个长辈为起点，把他的子孙后代像开枝散叶一样全部展示出来。这两个界面的成功渲染，从侧面证明了底层 Recursive CTE（递归公用表表达式）代码写得健壮。极其耗时的“层层找长辈、找晚辈”的循环回溯操作，被完全下推给了数据库引擎在后台默默完成。应用层只需要接收查好的扁平数据去驱动前端画图就行了，解决了万级复杂关系网带来的计算压力。

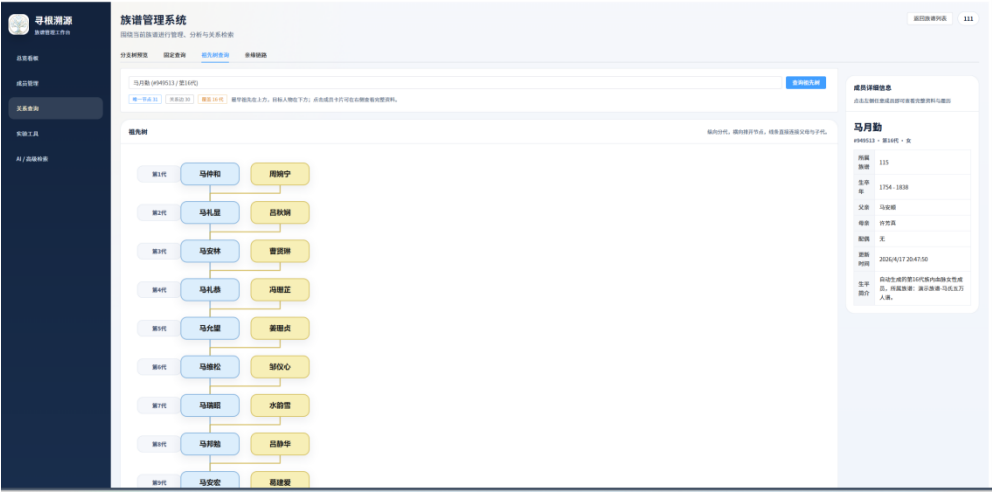


图5.4 祖先树查询图

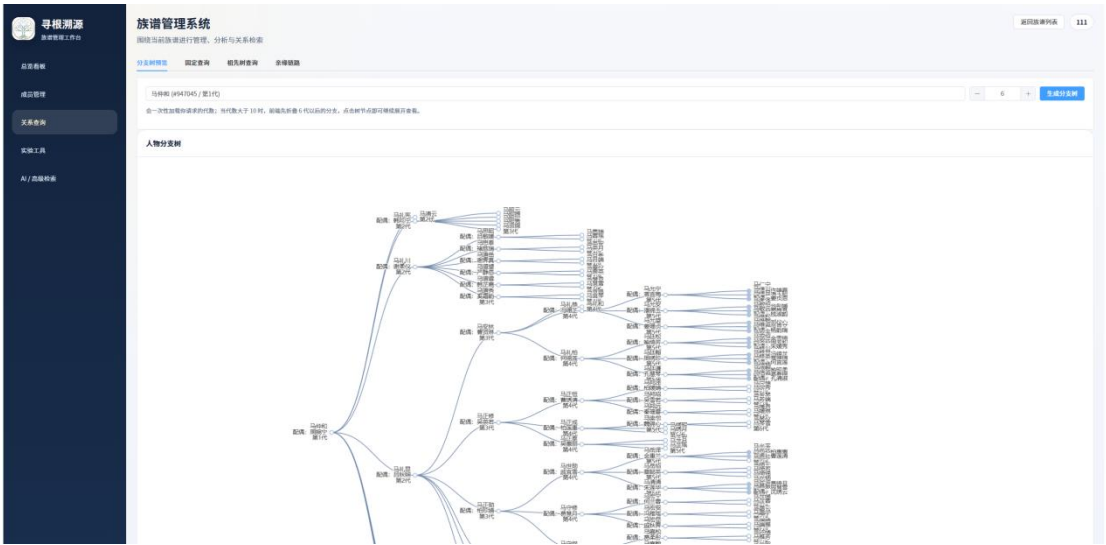


图5.5 分支树预览图

5.2.4 亲缘链路计算与索引寻址

最后是系统里最考验连通性运算的“亲缘链路”功能。这个模块可以查出族谱里任意两个人是怎么论称呼、怎么连通的。在测试中，我们选了深层第 14 代的一个节点和第 1 代的老祖宗发起寻路查询。从数据库的角度看，这其实就是在庞大的节点库里找最短路径。得益于我们在第三章提前建好的带有 `WHERE father_id IS NOT NULL` 等条件的部分复合索引，执行引擎在摸排时直接利用索引跳过了大量无关的孤立节点，在内存中收缩了搜索范围。最终，前端界面展示了中间隔着的二十多代直系亲属。在面对巨大的关系拓扑网络时，跨越几十代的链路查询依然能做到快速显示的结果，说明我们底层的索引优化是奏效了。

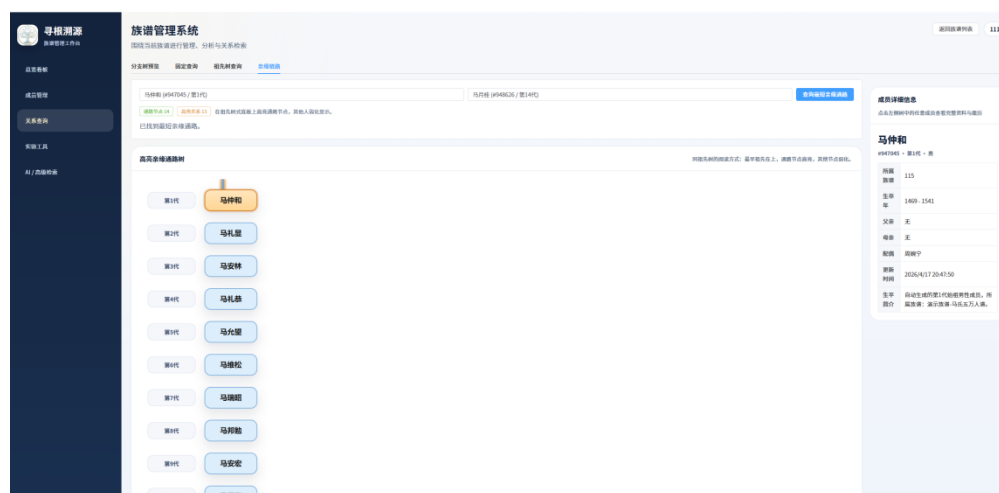


图5.6 最短亲缘链路图

6 个人工作内容及感想

在本次数据库课程设计中，我深度参与了系统的核心数据库架构设计、底层高级 SQL 编写以及存储过程的开发与联调工作。相比于以往在命令行里敲写简单的单表增删改查练习，这次实验让我彻底跳出了“表操作工”的局限，站在了系统整体架构的高度去审视数据库。我认识到，数据库绝不仅仅是一个被动存放数据的无脑仓库，更是整个业务逻辑运转和系统安全防御的坚实底座。

在具体的设计与开发过程中，我印象最深的是对家族树拓扑结构的物理映射与约束管理。族谱并不是一张简单的二维报表，而是一个充满自引用外键、多对多协作以及深层递归的有向图结构。这要求我们在建表时不能仅仅满足于范式的理论推导，更要考虑真实业务的容错性。例如，在处理成员表内部的父母与配偶关系时，我深刻认识到了级联删除（CASCADE）和置空保护（SET NULL）在不同业务场景下截然不同的破坏力与保护力。同时，利用底层触发器（如防断链保护和配偶双向同步）来接管复杂的业务逻辑，不仅保证了落库数据的绝对纯净，也极大地减轻了应用层代码的校验负担。

在整个项目的攻坚阶段，我也暴露出并克服了许多在查询调优方面的短板。在面对十万级数据的深层检索时，起初我认为只要语句能跑出结果即可，但发现简单的联表和普通的 LIKE 模糊搜索经常导致全表扫描，性能极差。经过查阅官方文档并不断利用 EXPLAIN 分析执行计划，我逐步掌握了利用 Recursive CTE（递归公用表表达式）解决深层树状结构遍历的技巧，也学会了如何运用部分复合索引以及 GIN 倒排字典联合 pg_trgm 扩展来突破模糊查询的性能瓶颈。这种从“只要能跑通就行”到“追求毫秒级响应延迟”的思维转变，是我在本次课设中最大的成长。

总体而言，这次课程设计是一次将《数据库系统》课堂上的抽象理论（如 BCNF 与 4NF 范式、关系代数、多值依赖）转化为真实工程成果的奇妙旅程。通过亲手设计表结构、推导数学范式、编写底层触发器与复杂索引，我不仅提升了编写原生高级 SQL 的硬核实力，更培养了严密的数据架构思维。这段实战经历，为我今后在软件开发和复杂系统架构设计领域打下了极其扎实且不可替代的基础。